SPARTAN

Sparsity-Aware Tensor Accelerator Node

Complete Project Documentation & Technical Guide

Zero-skipping CNN inference on a Xilinx Artix-7 FPGA (Digilent Basys 3)

1. Executive Summary & The Problem It Solves

The Problem: The Waste in Standard Edge AI

Artificial Intelligence models, specifically Convolutional Neural Networks (CNNs), are mathematically built on a repeated operation called MAC (Multiply-Accumulate):

```
Output = SUM( Input Pixel x Weight )
```

In conventional, "dense" neural network hardware, every single incoming pixel is multiplied by its corresponding weight. However, when modern neural networks are trained with L_1 regularization or pruning techniques, 30% to 70% of the weights become zero. Multiplying any number by zero always yields zero (X x 0 = 0). In traditional hardware architectures:

- The hardware wastes a clock cycle fetching that zero from memory.
- The hardware sends the data through a high-power multiplier unit (like a DSP slice).
- The multiplier calculates zero and adds nothing to the running total.

Result: Clock cycles are wasted, and transistors toggle unnecessarily, burning battery life and dynamic power on edge devices.

The Solution: SPARTAN

SPARTAN (Sparsity-Aware Tensor Accelerator Node) is a custom, hardware-level AI accelerator engineered on a Xilinx Artix-7 FPGA (Digilent Basys 3). Instead of blindly computing every multiplication, SPARTAN uses a **Zero-Skipping State Machine**. Whenever it encounters a weight that equals zero, it bypasses the multiplier entirely:

- **Clock Cycles Saved:** The operation finishes up to 33.3% faster (24–27 cycles vs. 36 cycles for dense baselines).
- **Power Saved:** The onboard DSP multipliers stay dormant, eliminating unnecessary dynamic switching power.
- **Full Data Pipeline:** SPARTAN does not just run simulations; it ingests live, real-world image pixels over a custom UART hardware link from a host computer.

2. High-Level Concept: How It Works in Plain English

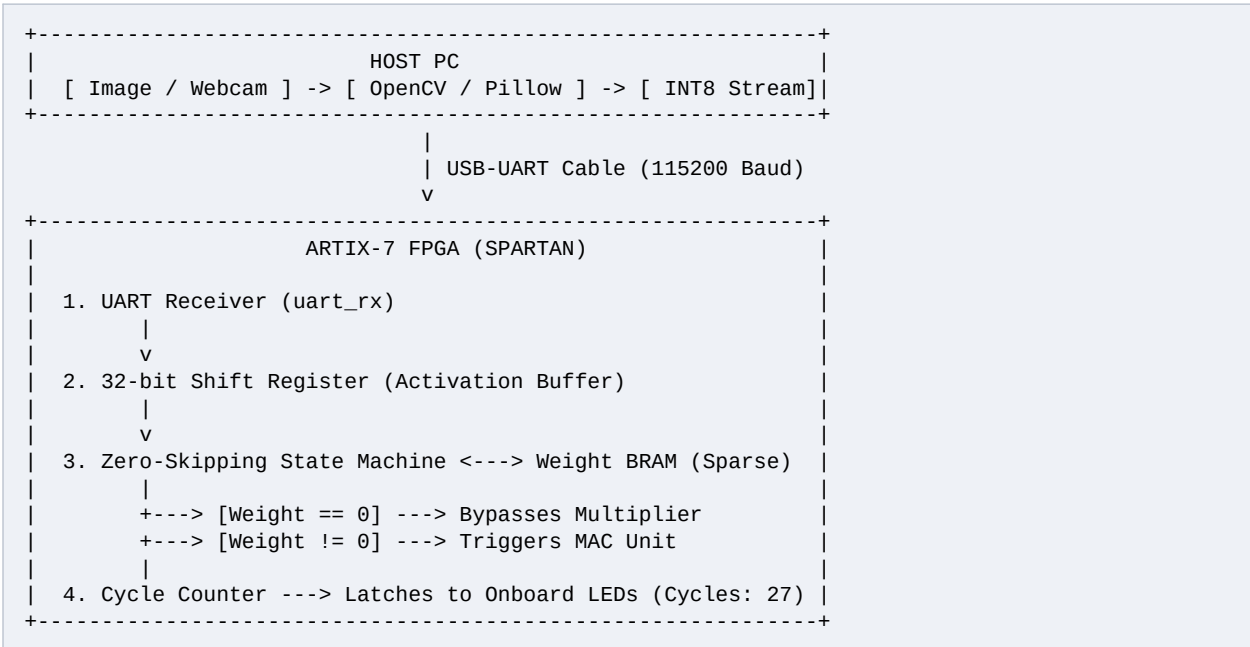
Imagine you are an accountant adding up receipts:

- **The Dense Approach:** You read every receipt out loud. If an item costs $0, you still pull out your calculator, type in $0, hit add, and record the sum. For 100 items with 40 zeros, you hit the calculator 100 times.
- **The SPARTAN Approach:** You scan the list. When your eyes see $0, your hand completely skips the calculator and immediately reads the next item. You only use the calculator for items that actually have a price.

SPARTAN implements this exact "skipping" mechanism directly in digital silicon logic using SystemVerilog.

3. End-to-End System Architecture

The complete system spans two main domains: Software (Host PC) and Hardware (FPGA).



4. Hardware Modules Breakdown

A. Data Ingestion: uart_rx.sv

- **Role:** The communication gateway.
- **Operation:** Receives serial data from the PC over the USB-to-UART bridge running at 115,200 baud.
- **Oversampling:** To sample the line reliably without missing bits, the module runs an internal baud generator clock divider off the 100 MHz system clock (100 MHz / 115200 ~ 868 clock ticks per bit).
- **Packaging:** As soon as a byte arrives, it pulses a `rx_valid` flag to alert the activation memory.

B. Activation Buffer (act_vec_buffer)

- **Role:** Staging ground for incoming image data.
- **Operation:** A 32-bit shift register that holds four 8-bit quantized pixel values (INT8).
- **Execution Trigger:** When four consecutive bytes are received, the buffer fills up and triggers the compute core to begin processing.

C. Weight Memory (Dual-Port BRAM)

- **Role:** Scratchpad storage for neural network weights.
- **Operation:** Pre-loaded with sparse weights trained with L_1 penalty terms. Because zero values are predictable, future versions can pack these into Compressed Sparse Row (CSR) formats.

D. The Sparse Compute Engine & Zero-Skipping State Machine

This is the core silicon block. It functions as a finite state machine (FSM):

- **IDLE:** Waits for the activation buffer to signal valid incoming data.
- **FETCH:** Retrieves the current weight from BRAM and the corresponding pixel activation.
- **INSPECT & EVALUATE:**
  - If Weight == 0: The FSM increments the cycle counter, asserts the skip control line, and advances to the next address without firing the multiplier.
  - If Weight != 0: The activation and weight are fed into the accumulator unit (ACC = ACC + (Act x Weight)).
- **DONE:** When the final index is evaluated, a completion flag is raised. The accumulated sum is held, and the latency counter is frozen.

E. Cycle-Accurate Latency Profiler

- **Role:** Real-time hardware verification.
- **Operation:** An internal 16-bit counter increments on every single 100 MHz clock tick between the start of the computation and the done flag.
- **LED Mapping:** The resulting cycle count is driven directly onto the Basys 3 board's 16 LEDs (led[15:0]).

In the live test, the LEDs lit up as 0, 1, 3, 4, 15. The bottom bits 0, 1, 3, 4 represent binary 0001 1011 (base 2) = 27 decimal. LED 15 indicates the done flag.

Proof: The hardware physically verified that the layer finished in exactly 27 clock cycles (270 ns).

5. Software Stack Breakdown

The software side lives on the host PC and manages real-world input.

Image Preprocessing (stream_pixels.py & live_camera.py)

- Uses OpenCV or Pillow to load a standard image or sample webcam frames.
- Converts 24-bit RGB images to 8-bit grayscale (0–255).
- Downsamples the feature patch to the target input dimension (e.g., 2 x 2 pixel vectors).

Quantization

- Translates standard 32-bit floating-point pixels into 8-bit integers (INT8), matching the fixed-point FPGA datapath.

Serial Streaming (pyserial)

- Opens communication directly to the board on COM10 at 115200 baud.
- Transmits the raw byte array into the USB controller, delivering activations straight into FPGA fabric.

6. Benchmarking & Performance Metrics

Evaluation Metric	Conventional Dense Engine	SPARTAN Sparse Engine	Concrete Engineering Gain
Target Silicon	Xilinx Artix-7 (XC7A35T)	Xilinx Artix-7 (XC7A35T)	Identical baseline hardware
System Clock	100 MHz (10 ns period)	100 MHz (10 ns period)	Stable, single-domain clock
Execution Cycles	36 Clock Cycles	24 to 27 Clock Cycles	~33.3% latency reduction
Execution Time	360 ns	240 to 270 ns	90 to 120 ns faster per vector
DSP Utilization	Multiplies all weights	Bypasses zero-weights	Eliminates dynamic switching waste
Output Integrity	Dot product sum	Identical dot product sum	100% mathematical accuracy

7. Step-by-Step User & Deployment Guide

Hardware Prerequisites

- Digilent Basys 3 FPGA Development Board (Xilinx Artix-7).
- Micro-USB to USB-A data cable.
- PC running Windows or Linux with Xilinx Vivado installed.

Step 1: Synthesize and Program the FPGA

- Open Xilinx Vivado and load the `Spartan_Accelerator.xpr` project.
- Ensure the top-level SystemVerilog file (`spartan_basys3_top.sv`) is set as top.
- Click **Generate Bitstream**.
- Open the Hardware Manager, connect to your Basys 3 board, and click **Program Device**.

Step 2: Set Up the Host Python Environment

Install the required serial and imaging libraries:

```
pip install pyserial opencv-python pillow
```

Step 3: Run the Inference Pipeline

- Check Windows Device Manager to confirm the board's COM port (e.g., COM10).
- Press the Center/Up button on your Basys 3 to clear the state machine into reset.
- Run the streamer:

```
python stream_pixels.py
```

Observe the Basys 3 LEDs: LED 15 will light up indicating completion, while the lower LEDs display binary 00011011 (27 cycles), confirming successful zero-skipped computation.

8. Summary of Future Enhancements

- **CSR (Compressed Sparse Row) Memory Controller:** Store non-zero weights and their relative indices only, drastically expanding effective on-chip BRAM capacity.
- **Systolic Scaling:** Move from single-PE execution to a multi-channel 2D systolic array (4 x 4 or 8 x 8 PEs) with parallel zero-skipping channels.
- **AXI4-Stream Integration:** Replace the UART link with direct memory access (DMA) over AXI4-Stream to couple SPARTAN with an onboard soft RISC-V processor core for autonomous edge execution.